Data Mining

Lecture Notes for Chapter 4

Artificial Neural Networks

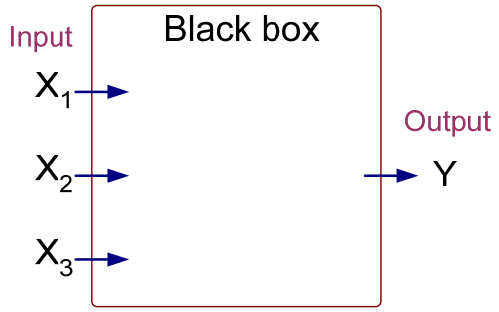
Introduction to Data Mining , 2nd Edition

by

Tan, Steinbach, Karpatne, Kumar

Artificial Neural Networks (ANN)

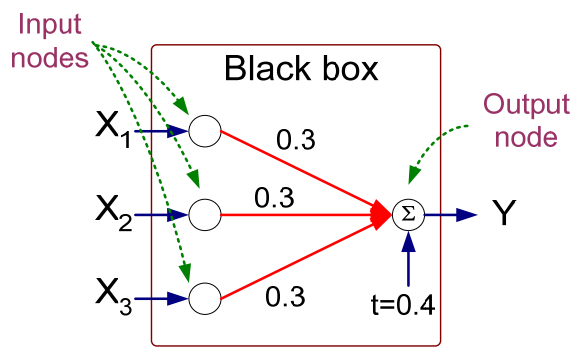
X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1



Output Y is 1 if at least two of the three inputs are equal to 1.

Artificial Neural Networks (ANN)

X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1

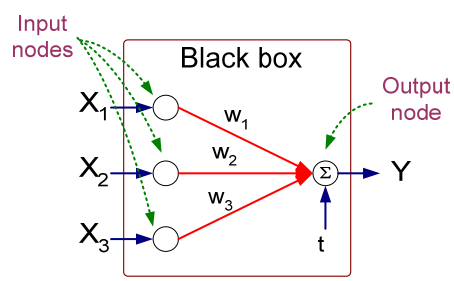


$$Y = sign\,(0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4)$$

$$\text{where } sign\,(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Artificial Neural Networks (ANN)

- Model is an assembly of inter-connected nodes and weighted links
- Output node sums up each of its input value according to the weights of its links
- Compare output node against some threshold t

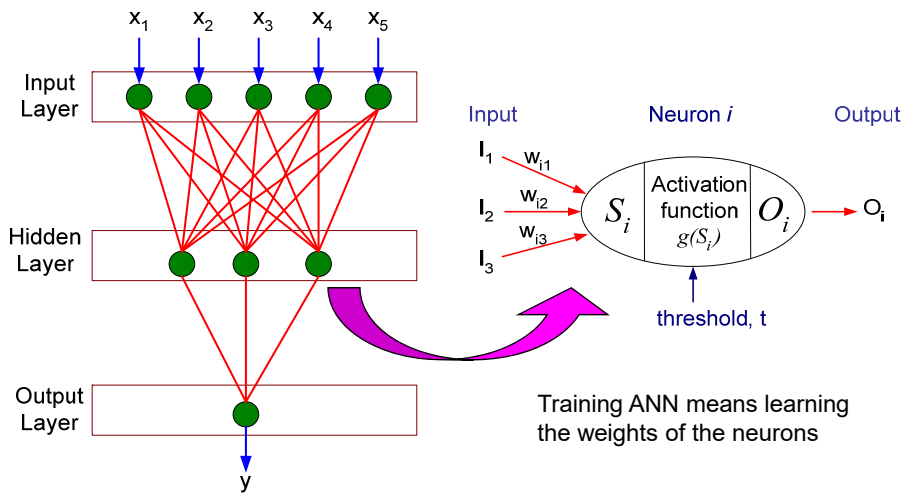


Perceptron Model

$$Y = sign(\sum_{i=1}^{d} w_i X_i - t)$$

$$= sign(\sum_{i=0}^{d} w_i X_i)$$

General Structure of ANN

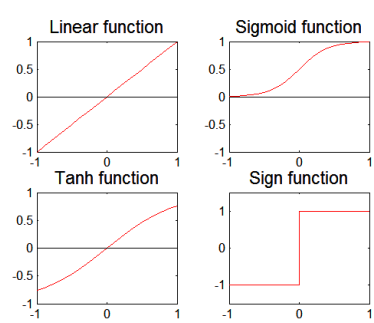


Training ANN means learning the weights of the neurons

Artificial Neural Networks (ANN)

- Various types of neural network topology
  - single-layered network (perceptron) versus multi-layered network
  - Feed-forward versus recurrent network
- Various types of activation functions (f)

$$Y = f(\sum_i w_i X_i)$$



Perceptron

- Single layer network
  - Contains only input and output nodes
- Activation function: f = sign(w•x)
- Applying model is straightforward

$$Y = sign(0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4)$$

$$\text{where } sign(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

  - $X_1 = 1$, $X_2 = 0$, $X_3 = 1$ => y = sign(0.2) = 1

Perceptron Learning Rule

- Initialize the weights (w_0, w_1, …, w_d)
- Repeat
  - For each training example (x_i, y_i)
    - Compute f(w, x_i)
    - Update the weights:

$$w^{(k+1)} = w^{(k)} + \lambda\left[y_i - f(w^{(k)}, x_i)\right]x_i$$

- Until stopping condition is met

Perceptron Learning Rule

- Weight update formula:

$$w^{(k+1)} = w^{(k)} + \lambda\left[y_i - f(w^{(k)}, x_i)\right]x_i \;;\; \lambda\text{: learning rate}$$

- Intuition:
  - Update weight based on error: $e = \left[y_i - f(w^{(k)}, x_i)\right]$
  - If y=f(x,w), e=0: no update needed
  - If y>f(x,w), e=2: weight must be increased so that f(x,w) will increase
  - If y<f(x,w), e=-2: weight must be decreased so that f(x,w) will decrease

Example of Perceptron Learning

$$w^{(k+1)} = w^{(k)} + \lambda\left[y_i - f(w^{(k)}, x_i)\right]x_i$$

$$Y = sign(\sum_{i=0}^{d} w_i X_i)$$

$$\lambda = 0.1$$

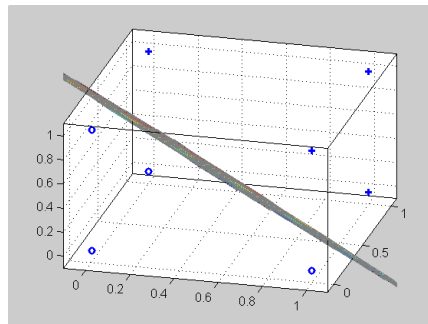
X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1

	w_0	w_1	w_2	w_3
0	0	0	0	0
1	-0.2	-0.2	0	0
2	0	0	0	0.2
3	0	0	0	0.2
4	0	0	0	0.2
5	-0.2	0	0	0
6	-0.2	0	0	0
7	0	0	0.2	0.2
8	-0.2	0	0.2	0.2

Epoch	w_0	w_1	w_2	w_3
0	0	0	0	0
1	-0.2	0	0.2	0.2
2	-0.2	0	0.4	0.2
3	-0.4	0	0.4	0.2
4	-0.4	0.2	0.4	0.4
5	-0.6	0.2	0.4	0.2
6	-0.6	0.4	0.4	0.2

Perceptron Learning Rule

- Since f(w,x) is a linear combination of input variables, decision boundary is linear



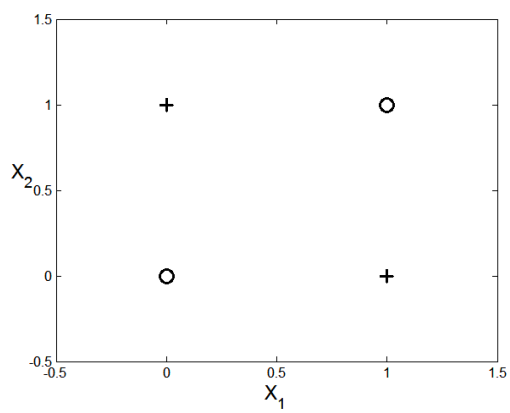
- For nonlinearly separable problems, perceptron learning algorithm will fail because no linear hyperplane can separate the data perfectly

Nonlinearly Separable Data

$$y = x_1 \oplus x_2$$

x_1	x_2	y
0	0	-1
1	0	1
0	1	1
1	1	-1

XOR Data

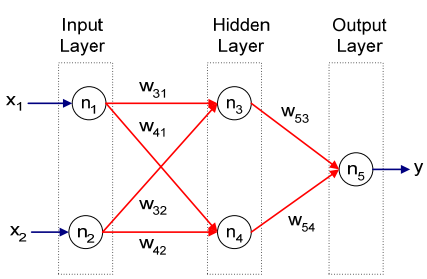


Multilayer Neural Network

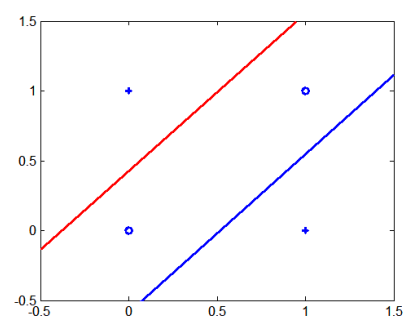
- Hidden layers
  - intermediary layers between input & output layers
- More general activation functions (sigmoid, linear, etc)

Multi-layer Neural Network

- Multi-layer neural network can solve any type of classification task involving nonlinear decision surfaces



XOR Data



Learning Multi-layer Neural Network

- Can we apply the perceptron learning rule to each node, including hidden nodes?
  - Perceptron learning rule computes error term e = y-f(w,x) and updates weights accordingly
    - Problem: how to determine the true value of y for hidden nodes?
  - Approximate error in hidden nodes by error in the output nodes
    - Problem:
      - Not clear how adjustment in the hidden nodes affect overall error
      - No guarantee of convergence to optimal solution

Gradient Descent for Multilayer NN

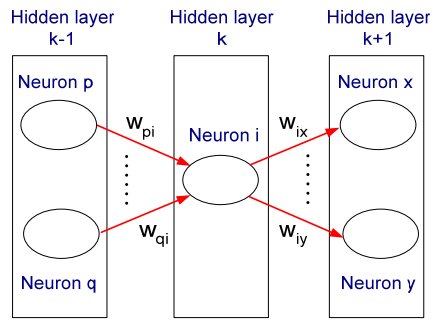
- Weight update: $$w_j^{(k+1)} = w_j^{(k)} - \lambda \frac{\partial E}{\partial w_j}$$
- Error function: $$E = \frac{1}{2}\sum_{i=1}^{N}\left(t_i - f(\sum_j w_j x_{ij})\right)$$
- Activation function f must be differentiable
- For sigmoid function:

$$w_j^{(k+1)} = w_j^{(k)} + \lambda \sum_i (t_i - o_i) o_i (1 - o_i) x_{ij}$$

- Stochastic gradient descent (update the weight immediately)

Gradient Descent for MultiLayer NN

- For output neurons, weight update formula is the same as before (gradient descent for perceptron)



- For hidden neurons:

$$w_{pi}^{(k+1)} = w_{pi}^{(k)} + \lambda o_i (1 - o_i) \sum_{j \in \Phi_i} \delta_j w_{ij} x_{pi}$$

$$\text{Output neurons}: \delta_j = o_j (1 - o_j)(t_j - o_j)$$

$$\text{Hidden neurons}: \delta_j = o_j (1 - o_j) \sum_{k \in \Phi_j} \delta_k w_{jk}$$

Design Issues in ANN

- Number of nodes in input layer
  - One input node per binary/continuous attribute
  - k or $\log_2$ k nodes for each categorical attribute with k values
- Number of nodes in output layer
  - One output for binary class problem
  - k or $\log_2$ k nodes for k-class problem
- Number of nodes in hidden layer
- Initial weights and biases

Characteristics of ANN

- Multilayer ANN are universal approximators but could suffer from overfitting if the network is too large
- Gradient descent may converge to local minimum
- Model building can be very time consuming, but testing can be very fast
- Can handle redundant attributes because weights are automatically learnt
- Sensitive to noise in training data
- Difficult to handle missing attributes

Recent Noteworthy Developments in ANN

- Use in deep learning and unsupervised feature learning
  - Seek to automatically learn a good representation of the input from unlabeled data
- Google Brain project
  - Learned the concept of a 'cat' by looking at unlabeled pictures from YouTube
  - One billion connection network

Deep Neural Networks

- Involve a large number of hidden layers
- Can represent features at multiple levels of abstraction
- Often require fewer nodes per layer to achieve generalization performance similar to shallow networks
- Deep networks have become the technique of choice for complex problems such as vision and language processing

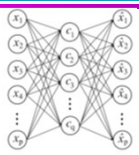
Deep Nets: Challenges and Solutions

- Challenges
  - Slow convergence
  - Sensitivity to initial values of model parameters
  - The larger number of nodes makes deep networks susceptible to overfitting
- Solutions
  - Large training data sets
  - Advances in computational power, e.g., GPUs
  - Algorithmic advances
    - New architectures and activation units
    - Better parameter and hyper-parameter selection
    - Regularization

Deep Learning Characteristics

- Pre-training allow deep learning models to reuse previous learning.
  - The learned parameters of the original task are used as initial parameter choices for the target task
  - Particularly useful when the target application has a smaller number of labeled training instances than the one used for pre-training
- Deep learning techniques for regularization help in reducing the model complexity
  - Lower model complexity promotes good generalization performance
  - The dropout method is one regularization approach
  - Regularization is especially important when we have
    - high-dimensional data
    - a small number of training labels
    - the classification problem is inherently difficult.

Deep Learning Characteristics …



Single-layer Autoencoder

- Using an autoencoder for pretraining can
  - Help eliminate irrelevant attributes
  - Reduce the impact of redundant attributes.
- ANN models, especially deep models, can find inferior and locally optimal solutions,
  - Deep learning techniques have been proposed to ensure adequate learning of an ANN
  - Example: Skip connections
- Specialized ANN architectures have been designed to handle various data sets.
  - Convolutional Neural Networks} (CNN) handle two-dimensional gridded data and are used for image processing
  - Recurrent Neural Network handles sequences and are used to process speech and language